Algorithms 2

Satisfiability via the Lovász Local Lemma

Daniel Rosenberg & Michael Trushkin

Ariel University

1. CNF Formulae with Few Repeating Variables

By a ***k*-CNF formula** we mean:

$$\underbrace{(x_1 \vee x_2 \vee x_3 \ldots)}_{=k} \wedge \underbrace{(\overline{x}_6 \vee x_2 \vee \overline{x}_{15} \ldots)}_{=k} \wedge \ldots$$

Each clause has exactly k literals.

By a ***k*-CNF formula** we mean:

$$\underbrace{(x_1 \vee x_2 \vee x_3 \ldots)}_{=k} \wedge \underbrace{(\overline{x}_6 \vee x_2 \vee \overline{x}_{15} \ldots)}_{=k} \wedge \ldots$$

Each clause has exactly k literals.

Theorem 2 Lemma Let φ be a k-CNF formula. If no variable of φ appears in more than $2^k/4k$ clauses, then φ is satisfiable.

Let m = # clauses of φ.

- Assign each variable of φ a value from $\{0, 1\}$ **u.a.r.** and independently.
- For $i \in [m]$, define:

$$\mathcal{E}_i := \text{the event that clause } i \text{ is unsatisfied}$$

We seek to prove $\mathbb{P}\left[\bigcap_{i \in [m]} \overline{\mathcal{E}_i}\right] > 0$ by appealing to the **Lovász local lemma**.

1.2 Proof: Setup

Let m = # clauses of φ.

- Assign each variable of φ a value from $\{0, 1\}$ **u.a.r.** and independently.
- For $i \in [m]$, define:

$$\mathcal{E}_i := \text{the event that clause } i \text{ is unsatisfied}$$

We seek to prove $\mathbb{P}\left[\bigcap_{i \in [m]} \overline{\mathcal{E}_i}\right] > 0$ by appealing to the **Lovász local lemma**.

Theorem 4 (Symmetric Local Lemma) Let $\mathcal{E}_1, \ldots, \mathcal{E}_n$ be events s.t.:

1. **(Symmetry)** $\mathbb{P}[\mathcal{E}_i] \leq p$ for all $i \in [n]$
2. **(Limited dependency)** $\Delta(D(\mathcal{E}_1, \ldots, \mathcal{E}_n)) \leq d$
3. **(Bound)** $e \cdot p \cdot (d + 1) \leq 1$ **(Or** $4 \cdot p \cdot d \leq 1$**)**

Then $\mathbb{P}\left[\bigcap_{i=1}^{n} \overline{\mathcal{E}_i}\right] > 0$.

1.3 Proof: Dependency Graph

Define a graph D s.t.:

- $V(D) := \{\mathcal{E}_i : i \in [m]\}$
- $E(D) :=$ place edge $(\mathcal{E}_i, \mathcal{E}_j)$ if clauses i and j **share common variables**

1.3 Proof: Dependency Graph

Define a graph D s.t.:

- $V(D) := \{\mathcal{E}_i : i \in [m]\}$
- $E(D) :=$ place edge $(\mathcal{E}_i, \mathcal{E}_j)$ if clauses i and j **share common variables**

Event $\mathcal{E}_i$ is mutually independent of all $\mathcal{E}_j$ where clauses i and j have **no common variables** $\Rightarrow D$ is a dependency graph.

1.3 Proof: Dependency Graph

Define a graph D s.t.:

- $V(D) := \{\mathcal{E}_i : i \in [m]\}$
- $E(D) :=$ place edge $(\mathcal{E}_i, \mathcal{E}_j)$ if clauses i and j **share common variables**

Event $\mathcal{E}_i$ is mutually independent of all $\mathcal{E}_j$ where clauses i and j have **no common variables** $\Rightarrow D$ is a dependency graph.

Bounding $\Delta(D)$:

- Each variable appears in at most $2^k/(4k)$ clauses; a clause has $\leq k$ distinct variables.

$$\Delta(D) = \arg\max_{C} \sum_{x \in C} [\#\text{ number of clauses that } x \text{ lies in}] \leq k \cdot \frac{2^k}{4k} = 2^{k-2}$$

Bounding p:

$$\mathbb{P}[\mathcal{E}_i] \leq \frac{1}{2^k} \quad \forall i \in [m]$$

since each clause has k literals.

Checking the LLL condition:

$$4 \cdot p \cdot d \leq 4 \cdot \frac{1}{2^k} \cdot 2^{k-2} = \frac{4}{4} \leq 1$$

By the Lovász local lemma: $\mathbb{P}\left[\bigcap_{i=1}^{m} \overline{\mathcal{E}_i}\right] > 0$. $\square$

2. A Randomised Algorithm

Observation 1 The Lovász local lemma offers **existential** results only.

Observation 2 The Lovász local lemma offers **existential** results only.

We are interested in **manipulating** the local lemma to obtain **constructive** results.

Observation 3 The Lovász local lemma offers **existential** results only.

We are interested in **manipulating** the local lemma to obtain **constructive** results.

Previous result: If every variable appears in $\underbrace{\leq 2^{\alpha k}}_{\text{say } \alpha \leq 1}$ clauses, then φ is satisfiable.

Question: For such a φ, can we **find** a satisfying assignment?

A **two-phase approach** is proposed:

Definition 1 **Phase I:** A partial random assignment is sampled.

Prove: With positive prob, the partial assignment extends into a satisfying assignment.

Definition 2 **Phase II:** a.a.s. a certain dependency graph on events related to **deferred** variables has only **small** connected components.

Used to find a satisfying assignment via **brute-force**.

A **two-phase approach** is proposed:

Definition 3 **Phase I:** A partial random assignment is sampled.

Prove: With positive prob, the partial assignment extends into a satisfying assignment.

Definition 4 **Phase II:** a.a.s. a certain dependency graph on events related to **deferred** variables has only **small** connected components.

Used to find a satisfying assignment via **brute-force**.

Theorem 6 For all sufficiently large even $k \in \mathbb{N}$ fixed, $\exists \alpha := \alpha(k)$ s.t.:

$\varphi :=$ k-CNF formula with m clauses, every variable appearing in $\leq 2^{\alpha k}$ clauses.

Then a satisfying assignment for φ can be found in **expected poly time** in m.

Throughout:

- φ is k-CNF, $k \geq 20$ fixed.
- Variables $x_1, ..., x_\lambda$; Clauses $C_1, ..., C_m$.
- Each variable appears in $\leq 2^{\alpha k}$ clauses.
- $\alpha > 0$ to be determined later.

Throughout:

- φ is k-CNF, $k \geq 20$ fixed.
- Variables $x_1, ..., x_\lambda$; Clauses $C_1, ..., C_m$.
- Each variable appears in $\leq 2^{\alpha k}$ clauses.
- $\alpha > 0$ to be determined later.

Assumptions:

- No clause contains a **complementary pair** of literals
- No clause contains **repeating** literals

Observation 5 $\forall$ clause: # literals = # variables.

3. Phase I: Partial Random Assignment

Assign values to **some** of $x_1, ..., x_\lambda$:

Algorithm 1: Phase-I

1: **for** each x_i in $x_1, ..., x_\lambda$ sequentially **do**
2: **if** x_i lies in no unsatisfied clause with precisely $k/2$ assigned variables **then**
3: Assign x_i a value from $\{0, 1\}$ u.a.r., independently
4: **else** do not assign x_i any value; proceed to next variable
5: **end**

Assign values to **some** of $x_1, ..., x_\lambda$:

Algorithm 2: Phase-I

```
for each x_i in x_1, ..., x_λ sequentially do
    if x_i lies in no unsatisfied clause with precisely k/2 assigned variables then
        Assign x_i a value from {0, 1} u.a.r., independently
    else do not assign x_i any value; proceed to next variable
end
```

Definition 6 Dangerous clause: A clause C is **dangerous** if precisely $k/2$ of its literals have been assigned a value, yet C remains unsatisfied.

Deferred variable: A variable not assigned a value by Phase I.

Surviving clause: A clause that is unsatisfied by the partial Phase I assignment.

Key structural facts:

- A surviving clause **always** has $\leq k/2$ fixed variables.

Proof: Suppose C has $> k/2$ fixed vars. Then at some earlier point C had **exactly** $k/2$ fixed vars while still unsatisfied — so C was **dangerous** at that moment. But once dangerous, no further variables of C are ever assigned.

Contradiction. $\square$

Key structural facts:

- A surviving clause **always** has $\leq k/2$ fixed variables.

Proof: Suppose C has $> k/2$ fixed vars. Then at some earlier point C had **exactly** $k/2$ fixed vars while still unsatisfied — so C was **dangerous** at that moment. But once dangerous, no further variables of C are ever assigned. Contradiction. □

Observation 7 If C is a surviving clause then it has at least $k/2$ **deferred** variables.

Proof: By definition every unassigned variable is deferred. Since $f \leq k/2$ (proved above), C has $k - f \geq k/2$ deferred variables. □

4. Extendability into a Satisfying Assignment

Lemma 7 For all even sufficiently large fixed $k \in \mathbb{N}$, $\exists \alpha > 0$ s.t. there is an assignment to the deferred variables satisfying **all** surviving clauses.

Proof. We apply the local lemma.

Source of randomness (not the Phase I assignment!): Assign each **deferred** variable a value u.a.r. from $\{0, 1\}$ independently.

For a surviving clause C, define:

$$\mathcal{E}_C := C \text{ not satisfied by the random assignment}$$

4.2 Dependency Graph for Deferred Variables

Define graph D:

- $V(D) = \{\mathcal{E}_C : C \text{ a surviving clause}\}$
- $\mathcal{E}_C$ and $\mathcal{E}_{C'}$ adjacent if C and C' share a common **deferred** variable
- D is a dependency graph (same argument as before)

Theorem 8 (Symmetric Local Lemma) Let $\mathcal{E}_1, \ldots, \mathcal{E}_n$ be events s.t.:

1. **(Symmetry)** $\mathbb{P}[\mathcal{E}_i] \leq p$ for all $i \in [n]$
2. **(Limited dependency)** $\Delta(D(\mathcal{E}_1, \ldots, \mathcal{E}_n)) \leq d$
3. **(Bound)** $e \cdot p \cdot (d+1) \leq 1$ (Or $4 \cdot p \cdot d \leq 1$)

Then $\mathbb{P}\left[\bigcap_{i=1}^{n} \overline{\mathcal{E}_i}\right] > 0$.

4.2 Dependency Graph for Deferred Variables

Define graph D:

- $V(D) = \{\mathcal{E}_C : C \text{ a surviving clause}\}$
- $\mathcal{E}_C$ and $\mathcal{E}_{C'}$ adjacent if C and C' share a common **deferred** variable
- D is a dependency graph (same argument as before)

Bounding $\mathbb{P}[\mathcal{E}_C]$: Since C has $\geq k/2$ deferred vars:

$$\mathbb{P}[\mathcal{E}_C] \leq 2^{-k/2}$$

Bounding $\Delta(D)$:

- Each deferred var in C lies in $\leq 2^{\alpha k}$ other surviving clauses
- C has $\leq k$ deferred variables

$$\Delta(D) \leq k \cdot 2^{\alpha k}$$

Theorem 9 (Symmetric Local Lemma) Let $\mathcal{E}_1, \ldots, \mathcal{E}_n$ be events s.t.:

1. **(Symmetry)** $\mathbb{P}[\mathcal{E}_i] \leq p$ for all $i \in [n]$
2. **(Limited dependency)** $\Delta(D(\mathcal{E}_1, \ldots, \mathcal{E}_n)) \leq d$
3. **(Bound)** $e \cdot p \cdot (d+1) \leq 1$ (Or $4 \cdot p \cdot d \leq 1$)

Then $\mathbb{P}\left[\bigcap_{i=1}^{n} \overline{\mathcal{E}_i}\right] > 0$.

4.2 Dependency Graph for Deferred Variables

Define graph D:

- $V(D) = \{\mathcal{E}_C : C \text{ a surviving clause}\}$
- $\mathcal{E}_C$ and $\mathcal{E}_{C'}$ adjacent if C and C' share a common **deferred** variable
- D is a dependency graph (same argument as before)

Theorem 10 (Symmetric Local Lemma) Let $\mathcal{E}_1, \ldots, \mathcal{E}_n$ be events s.t.:
1. **(Symmetry)** $\mathbb{P}[\mathcal{E}_i] \leq p$ for all $i \in [n]$
2. **(Limited dependency)** $\Delta(D(\mathcal{E}_1, \ldots, \mathcal{E}_n)) \leq d$
3. **(Bound)** $e \cdot p \cdot (d+1) \leq 1$ (Or $4 \cdot p \cdot d \leq 1$)

Then $\mathbb{P}\left[\bigcap_{i=1}^{n} \overline{\mathcal{E}_i}\right] > 0$.

Bounding $\mathbb{P}[\mathcal{E}_C]$: Since C has $\geq k/2$ deferred vars:

$$\mathbb{P}[\mathcal{E}_C] \leq 2^{-k/2}$$

Bounding $\Delta(D)$:

- Each deferred var in C lies in $\leq 2^{\alpha k}$ other surviving clauses
- C has $\leq k$ deferred variables

$$\Delta(D) \leq k \cdot 2^{\alpha k}$$

LLL condition:

$$4 \cdot \Delta(D) \cdot p \leq e \cdot k \cdot 2^{\alpha k} \cdot 2^{-k/2} = k \cdot 2^{\alpha k + 2 - k/2} \leq 1$$

for k sufficiently large and α sufficiently small.

Local lemma asserts: $\mathbb{P}\left[\bigcap_{C \text{ surv.}} \overline{\mathcal{E}_C}\right] > 0$. $\square$

5. Component Sizes

5.1 The Plan

Let D be the dependency graph from the last proof.

- A connected component of D defines a **sub-formula** of φ
- φ has m clauses $\Rightarrow$ at most m components/sub-formulae

Let D be the dependency graph from the last proof.

- A connected component of D defines a **sub-formula** of φ
- φ has m clauses $\Rightarrow$ at most m components/sub-formulae

Observation 9 If all components have size $O(\log m)$, then at most $O(k \log m)$ variables are involved.

If φ is satisfiable, a brute-force search over all $2^{O(k \log m)} = m^{O(k)}$ options finds a satisfying assignment.

Current goal: Use Phase I to prove that a.a.s.
all components of D are "small".

Lemma 12 (Component Size): $\forall k$ as before $\exists \alpha = \alpha(k) > 0$ and $C := C(k, \alpha) > 0$ s.t. a.a.s. all components of D have size $\leq C \ln m$.

5.2 A Second Graph D'

Introduce a **deterministic** auxiliary graph D':

- $V(D') :=$ **all** clauses of φ (not just surviving ones)
- C and C' adjacent if they share common variables
- A vertex C of D' is a vertex of D iff C survived Phase I

Introduce a **deterministic** auxiliary graph D':

- $V(D') :=$ **all** clauses of φ (not just surviving ones)
- C and C' adjacent if they share common variables
- A vertex C of D' is a vertex of D iff C survived Phase I

Since $D \subseteq D'$:

$$\Delta(D) \leq \Delta(D') \leq k \cdot 2^{\alpha k} =: \Delta$$

Why D'? To analyse components of D, it suffices to analyse which subgraphs of D' survive Phase I.

Definition 7 4-tree of R: Let R be a connected subgraph of D'. A **4-tree** of R is a rooted tree S (need not be a subgraph of R) satisfying:

1. $V(S) \subseteq V(R)$
2. Any two nodes of S are at distance ≥ 4 in D'
3. Two nodes are adjacent in S if their D'-distance is **precisely** 4
4. Every vertex of R is either in S or at distance ≤ 3 from S in R

Definition 10 4-tree of R: Let R be a connected subgraph of D'. A **4-tree** of R is a rooted tree S (need not be a subgraph of R) satisfying:

1. $V(S) \subseteq V(R)$
2. Any two nodes of S are at distance ≥ 4 in D'
3. Two nodes are adjacent in S if their D'-distance is **precisely** 4
4. Every vertex of R is either in S or at distance ≤ 3 from S in R

Claim : Claim A Let S be a 4-tree of some connected subgraph R of D'. Then:

$$\mathbb{P}[V(S) \subseteq V(D)] \leq \left((\Delta + 1) \cdot 2^{-k/2}\right)^{v(S)}$$

Claim : Claim B Let R be a connected subgraph of D', S a u-tree of R of **maximum size**. Then:

$$v(S) \geq v(R)/\Delta^3$$

Strategy:

Observation 10 By Claim B: if any connected subgraph R of D' of size $\geq C \ln m$ survives in D, then it has a u-tree of size $\geq C \ln m/\Delta^3$ that also survived.

Strategy:

Observation 11 By Claim B: if any connected subgraph R of D' of size $\geq C \ln m$ survives in D, then it has a u-tree of size $\geq C \ln m/\Delta^3$ that also survived.

By Claim A: prove that a.a.s.
D' has **no** u-tree of size $\geq C \ln m/\Delta^3$ that survived in D.

$\Rightarrow$ No connected subgraph of D' of size $\geq C \ln m$ survives in D (a.a.s.) $\Rightarrow$ all components of D have size $\leq C \ln m$ (a.a.s.).

Proof. For any clause C:

$$\begin{aligned}\mathbb{P}[C \text{ survives}] &\leq \mathbb{P}[C \text{ is dangerous}] + \mathbb{P}[\geq 1 \text{ neighbour in } D' \text{ is dangerous}] \\ &\leq 2^{-k/2} + \Delta \cdot 2^{-k/2} = (\Delta + 1) \cdot 2^{-k/2}\end{aligned}$$

Proof. For any clause C:

$$\begin{aligned}\mathbb{P}[C \text{ survives}] &\leq \mathbb{P}[C \text{ is dangerous}] + \mathbb{P}[\geq 1 \text{ neighbour in } D' \text{ is dangerous}] \\ &\leq 2^{-k/2} + \Delta \cdot 2^{-k/2} = (\Delta + 1) \cdot 2^{-k/2}\end{aligned}$$

Key observation: Clauses at distance 2 in D' share **no** common variables $\Rightarrow$ the events that such clauses are dangerous are **mutually independent**.

Fix $u \in V(S)$. Any vertex of R that can cause u to be dangerous is at distance 1 from u, hence at distance ≥ 2 from all other members of $V(S)$.

$\Rightarrow$ The survival events of $V(S)$ are **mutually independent**.

$\Rightarrow$

$$\mathbb{P}[V(S) \subseteq V(D)] \leq \left((\Delta + 1) \cdot 2^{-k/2}\right)^{v(S)}. \quad \square$$

Proof. Suppose for contradiction that $v(S) < v(R)/\Delta^3$.

By definition, every member of $V(R)$ is at distance ≤ 3 from $V(S)$ in R. Fix $v \in V(D')$:

$$\begin{aligned} \#\text{ vxs in } D' \text{ at dist.} \leq 3 \text{ from } v &\leq \Delta + \Delta(\Delta-1) + \Delta(\Delta-1)(\Delta-2) \\ &= \Delta + \Delta^2 - \Delta + \Delta^3 - 3\Delta^2 + 2\Delta \\ &\leq \Delta^3 - 1 \end{aligned}$$

Therefore:

$$v(R) \leq |V(S)| \cdot (\Delta^3 - 1) < \frac{v(R)}{\Delta^3} \cdot (\Delta^3 - 1) < v(R). \quad \text{contradiction.} \quad \square$$

5.7 Component Size: Proof

Set $r := C \ln m$ for comfort. We need to bound $\mathbb{P}[\mathcal{E}]$ where $\mathcal{E} :=$ "a u-tree of size $\geq r/\Delta^3$ survived in D".

# of u-trees in D' of size r/Δ^3:

$$\leq m \cdot \Delta^{4r/\Delta^2}$$

- $\leq m$ ways to choose the root
- # ways to build a u-tree $\leq$ # Euler tours on r/Δ^3 nodes starting/ending at root
- Each edge of the u-tree represents a path of length u; at each node-visit, $\leq \Delta^u$ continuation options

5.7 Component Size: Proof

Set $r := C \ln m$ for comfort. We need to bound $\mathbb{P}[\mathcal{E}]$ where $\mathcal{E}$:= "a u-tree of size $\geq r/\Delta^3$ survived in D".

# of u-trees in D' of size r/Δ^3:

$$\leq m \cdot \Delta^{4r/\Delta^2}$$

- $\leq m$ ways to choose the root
- # ways to build a u-tree $\leq$ # Euler tours on r/Δ^3 nodes starting/ending at root
- Each edge of the u-tree represents a path of length u; at each node-visit, $\leq \Delta^u$ continuation options

By Claim A and the union bound:

$$\begin{aligned}
\mathbb{P}[\mathcal{E}] &\leq m \cdot \Delta^{4r/\Delta^2} \cdot \left((\Delta+1) \cdot 2^{-k/2}\right)^{r/\Delta^3} \\
&= \exp\left(\ln m + 4\frac{r}{\Delta^2}\ln\Delta + \frac{r}{\Delta^3}\ln(\Delta+1) - kr/(2\Delta^3)\right) \\
&\leq \exp(\ln m + (4r\ln(2k))/\Delta^2 + (r\ln(4k))/\Delta^3 - kr/(2\Delta^3)) \\
&= o(1)
\end{aligned}$$

by choosing α small enough and C large enough. $\square$

6. Grand Finale

Theorem 13 For all suff. large even $k \in \mathbb{N}$ fixed, $\exists \alpha := \alpha(k)$ s.t.:

$\varphi :=$ k-CNF with m clauses, every variable in $\leq 2^{\alpha k}$ clauses.

Then a satisfying assignment for φ can be found in **expected poly. time** in m.

Proof:

- **Phase I** a.a.s. decomposes φ into components of size $\leq C \ln m$ (Component Size Lemma)
- **Geometric experiment:** We expect $O(1)$ applications of Phase I to yield a valid decomposition
- **Extendability Lemma:** whatever decomposition we obtain is extendable into a satisfying assignment
- **Brute force on each component:** within expected time $m^{O(k)}$ we trace a satisfying assignment $\square$